

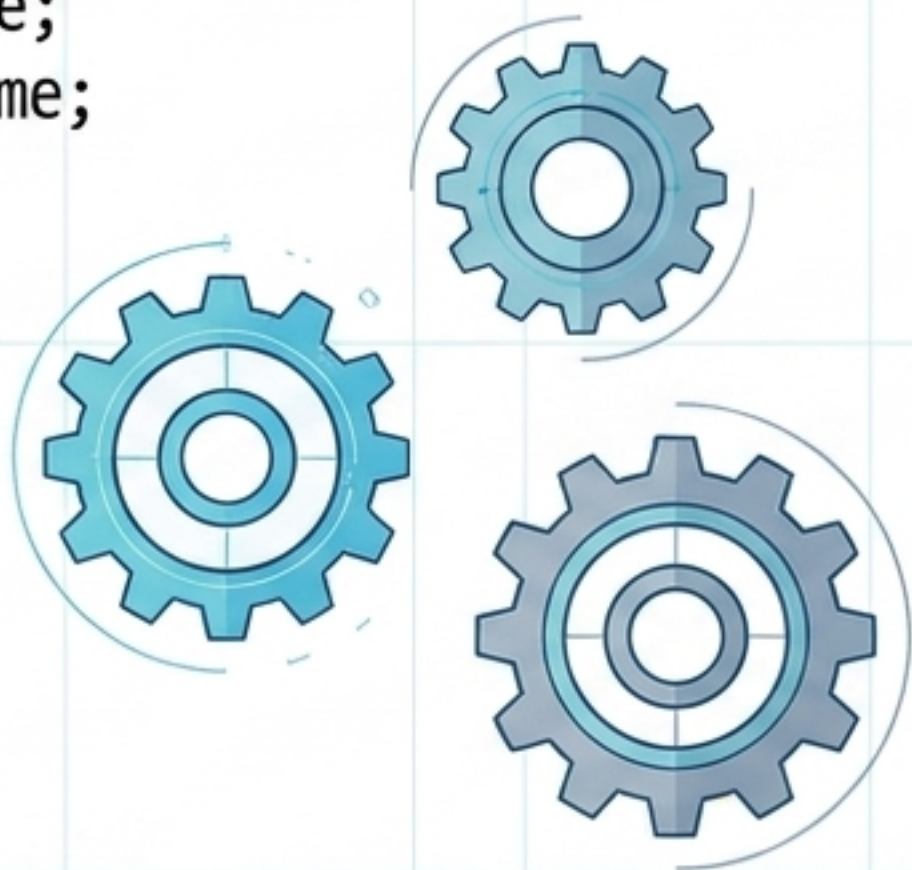
C# 디지털 설계도와 공구함 완벽 가이드

클래스의 기초부터 Random, List, Math 내장 유틸리티의 실전 활용까지

파편화된 데이터의 한계: 왜 변수만으로는 부족할까?

차량 한 대 (변수 3개 필요)

```
int carNumber;  
int inTime;  
int outTime;
```

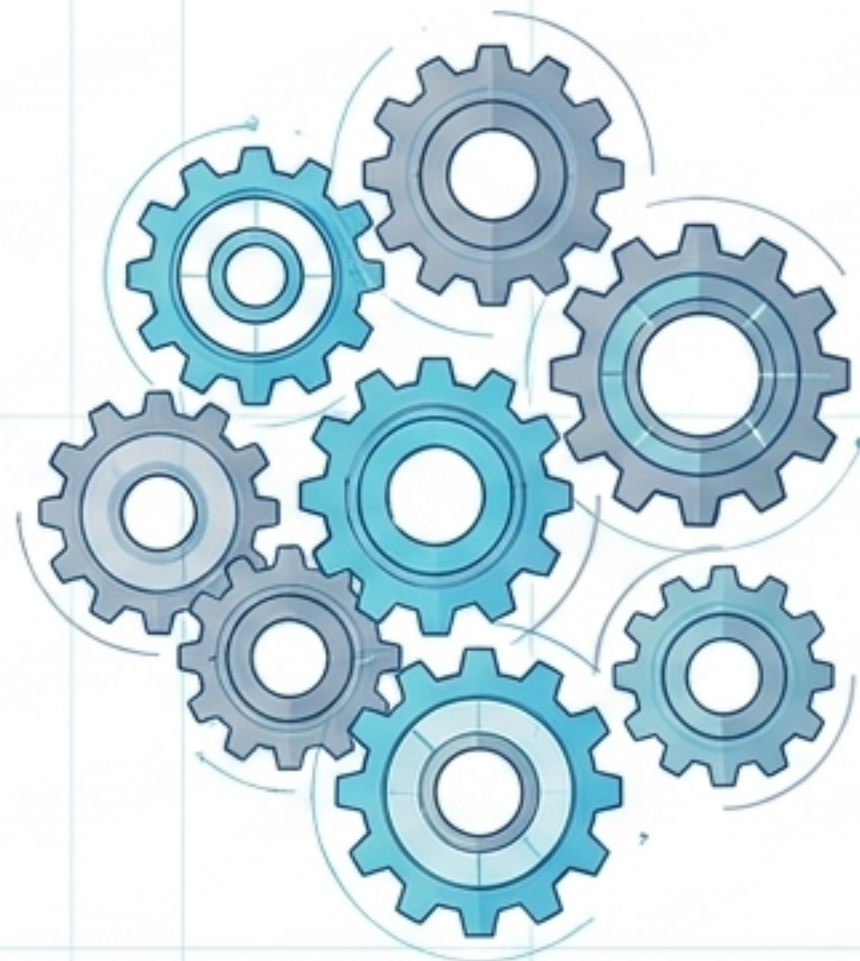


차량 세 대 (변수 9개 필요) - 관리 불가능

```
int carNumberA;  
int inTimeA;  
int outTimeA;
```

```
int carNumberB;  
int inTimeB;  
int outTimeB;
```

```
int carNumberC;  
int inTimeC;  
int outTimeC;
```

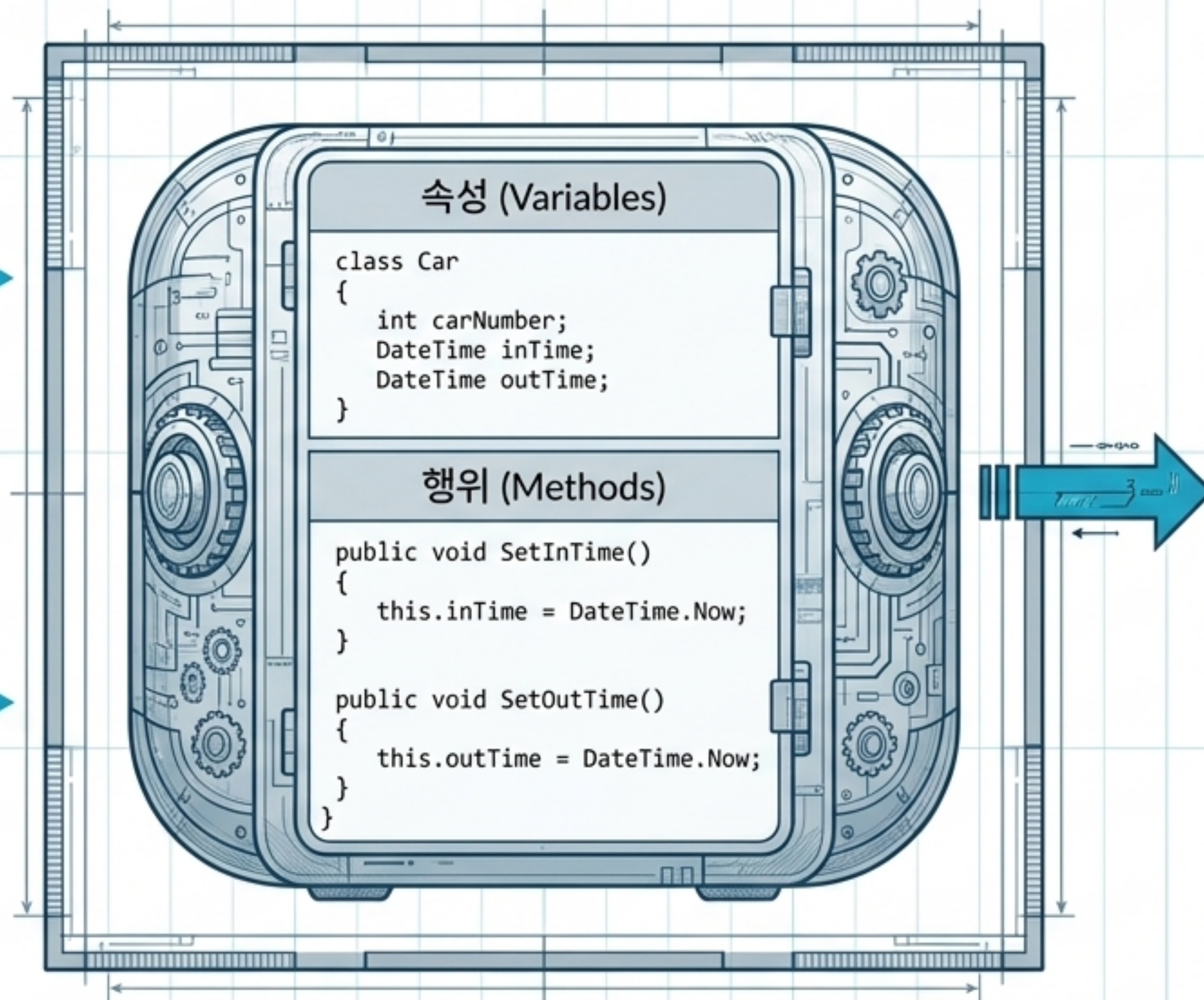


주차 관리 시스템에서 차량 번호(carNumber), 입차(inTime), 출차(outTime) 데이터는 항상 함께 움직여야 합니다. 개별 변수로 쪼개면 데이터의 맥락이 끊어집니다.

클래스(Class): 데이터와 동작을 묶어내는 단 하나의 설계도

사용자 정의 자료형:
개발자가 직접 만드는
새로운 데이터 규격.

지능형 캡슐:
단순히 데이터를 담는
것을 넘어, 스스로 동작
(메서드)할 수 있는 구조.



`Car[] cars = new Car[10];`

⚙️ “배열이 단순한 서랍장이라면, 클래스는 맞춤형 금고입니다.”

설계도를 실체화하는 마법의 주문

Car car = new Car();

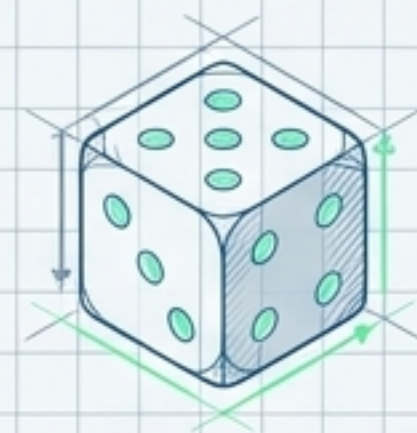
클래스 (Class):
우리가 만든
설계도(자료형)의 이름.

인스턴스 (Instance):
메모리에 실제로
만들어진 객체의 식별자.

키워드 (Keyword):
설계도를 바탕으로
실체를 생성하라는
시스템 명령어.

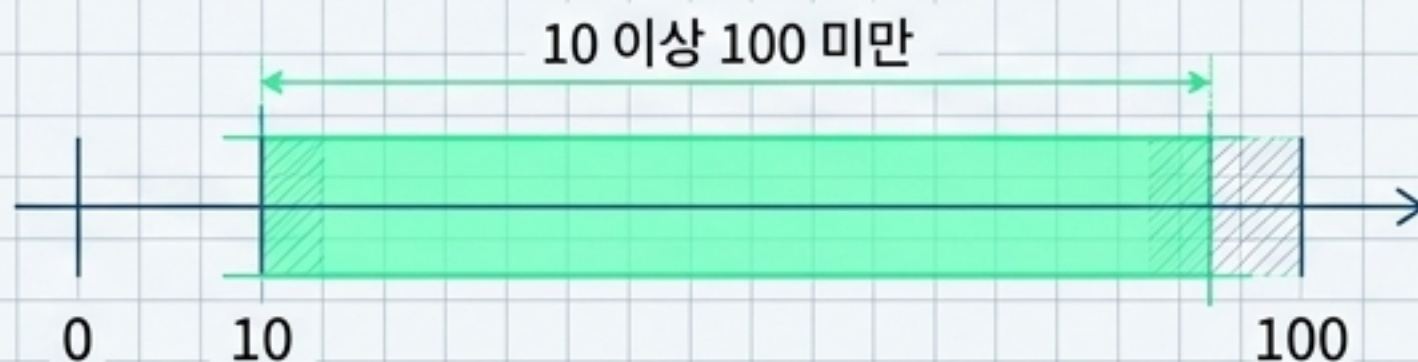
생성자 (Constructor):
클래스 이름과 동일한
형태를 띠며, 초기 조립을
담당하는 메서드.

도구함 #1. Random 클래스: 예측 불가능성을 통제하는 주사위



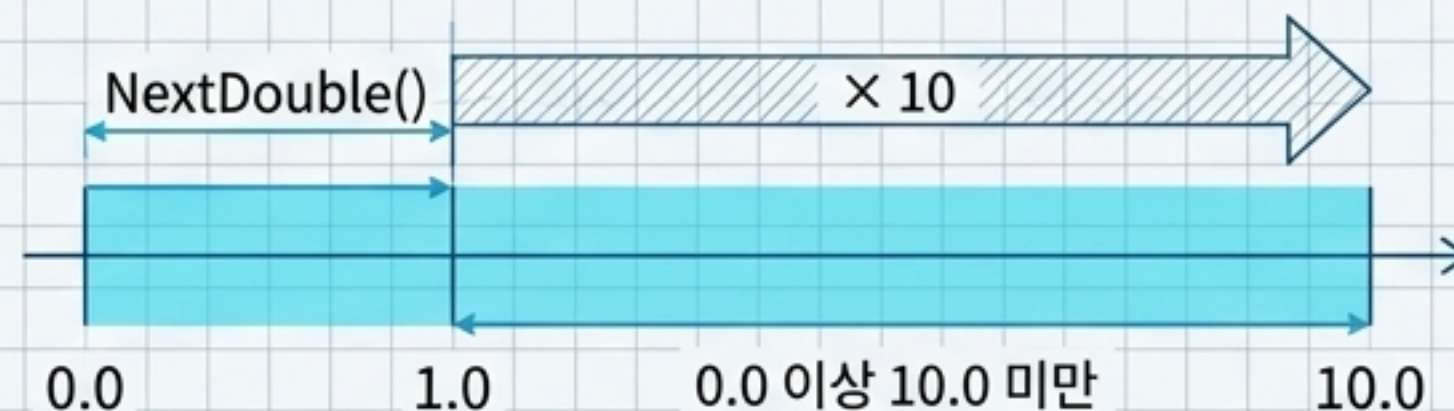
정수 난수 생성

```
Random random = new Random();  
Console.WriteLine(random.Next(10, 100));
```



실수 난수 생성

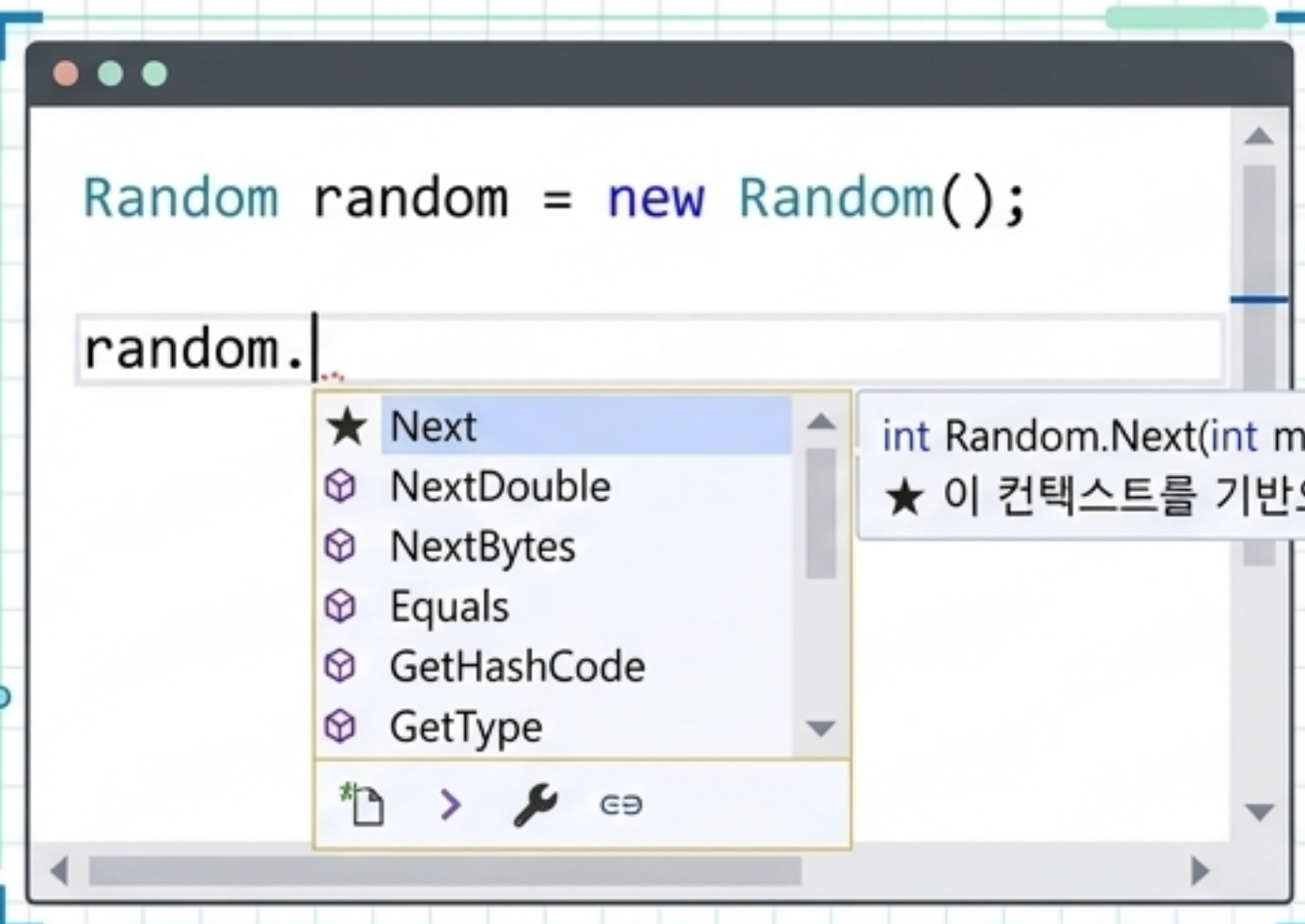
```
Random random = new Random();  
Console.WriteLine(random.NextDouble() * 10);
```



개발자의 인지적 부하를 덜어주는 스마트 어시스턴트

점(.)의 마법:

인스턴스 뒤에 '.'을 찍는 순간
사용 가능한 모든 도구 목록이
즉시 펼쳐집니다.

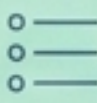
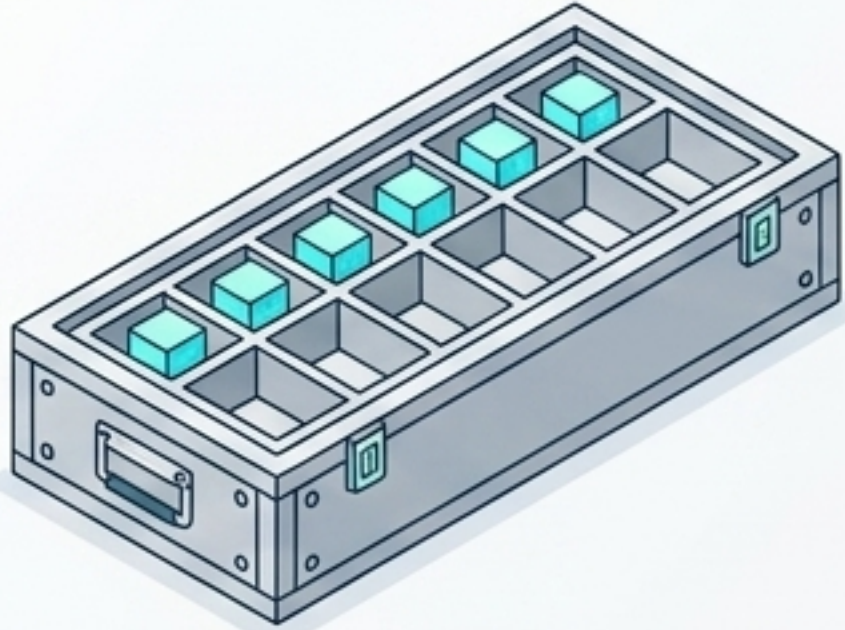
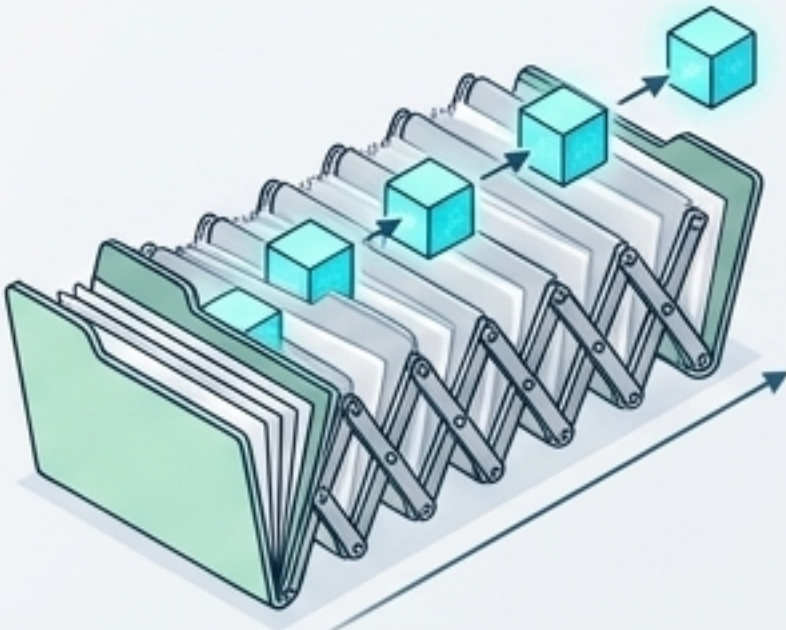


오버로드 탐색:

위/아래 방향키를 눌러 매개변수(입력값)의 형태가 다른 동일한 이름의 메서드들을 확인할 수 있습니다.

핵심 인사이트: 모든 코드를 외울 필요는 없습니다. 자동 완성 기능이 제시하는 컨텍스트를 읽고 유추하는 능력이 더 중요합니다.

도구함 #2. Array(배열) vs List(리스트)

 배열 - Array	 리스트 - List
 <pre data-bbox="333 1125 1532 1388">int[] intArray = new int[10]; long[] longArray = new long[10]; string[] stringArray = new string[10];</pre>	 <pre data-bbox="1799 1125 2998 1388">List<int> list = new List<int>();</pre>
• 크기 고정형 공간 • 최초 생성 시 공간을 지정하면 절대 변하지 않음 • 데이터 추가/삭제 전용 메서드 없음	• 가변형 제네릭 컨테이너 • 데이터가 들어오는 대로 공간이 무한히 확장됨 • 제네릭 <T>를 통해 특정 자료형만 받도록 필터링

유연하게 늘어나고 줄어드는 동적 컨테이너의 움직임

초기화 (Initialization)

```
new List<int>() { 52, 273, 32, 64 }
```



Add()

```
list.Add(100);
```



Remove(52)

```
list.Remove(52);
```



참조 누락 오류(CS0246)를 돌파하는 트러블슈팅 가이드

에러 발생

```
List<int> list = new List<int>();
```

붉은 밑줄. 도구 상자의 문이 닫혀있는 상태입니다.

해결 단축키

Ctrl + .

에러가 난 곳에 커서를 두고 단축키 입력 (또는 전구 아이콘 클릭).

참조 추가

```
using System.Collections.Generic;  
using System.Collections.Generic;  
using System.Cath  
using System  
using System.Enigth  
using System.Cainpitors  
using Utility  
using Linertiters
```

최상단에 참조를 추가하면 오류가 즉시 해결됩니다.



도구함 #3. Math 클래스: 인스턴스가 필요 없는 공용 유틸리티 보관함



Math.Abs():
절대값을 구합니다.

Math.Max() / Min():
두 매개변수 중 큰 값/
작은 값을 구합니다.

Math.Ceiling() / Floor()
/ Round(): 올림 / 내림 /
반올림을 처리합니다.

```
static void Main(string[] args)
{
    Console.WriteLine(Math.Abs(-52273));
    Console.WriteLine(Math.Ceiling(52.273));
    Console.WriteLine(Math.Floor(52.273));
    Console.WriteLine(Math.Max(52, 273));
    Console.WriteLine(Math.Min(52, 273));
    Console.WriteLine(Math.Round(52.273));
}
```

핵심 포인트: Math 클래스는 new 키워드로 인스턴스를 만들지 않고, 클래스 이름 뒤에 직접 점('.')을 찍어 사용합니다.

C# 멤버 호출의 두 가지 결정적 패러다임

인스턴스 멤버 (Instance Member)



대상: Random, List

특징: new 키워드로 나만의 복제본(인스턴스)을 메모리에 올려야만 사용 가능.

```
Random random = new Random();  
random.Next();
```

클래스 멤버 (Class Member)



대상: Math

특징: 원본 설계도 자체에 기능이 내장되어 있어 복제본 생성이 불필요. 공공재처럼 바로 접근.

```
Math.Abs(-50);  
Math.PI;
```

설계의 통찰: 낯선 클래스를 마주했을 때의 행동 강령

이 클래스는 내부적으로 고유한
'상태(State)'를 기억해야 하는가?

상태 유지형 (Stateful)

List처럼 내부 데이터를 누적해서 보관해야 하거나, Random처럼 고유한 시드(Seed)값을 가져야 하는 경우.

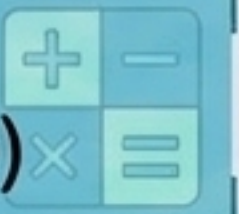
인스턴스 생성 필요
(new 키워드 사용)



기능 제공형 (Stateless)

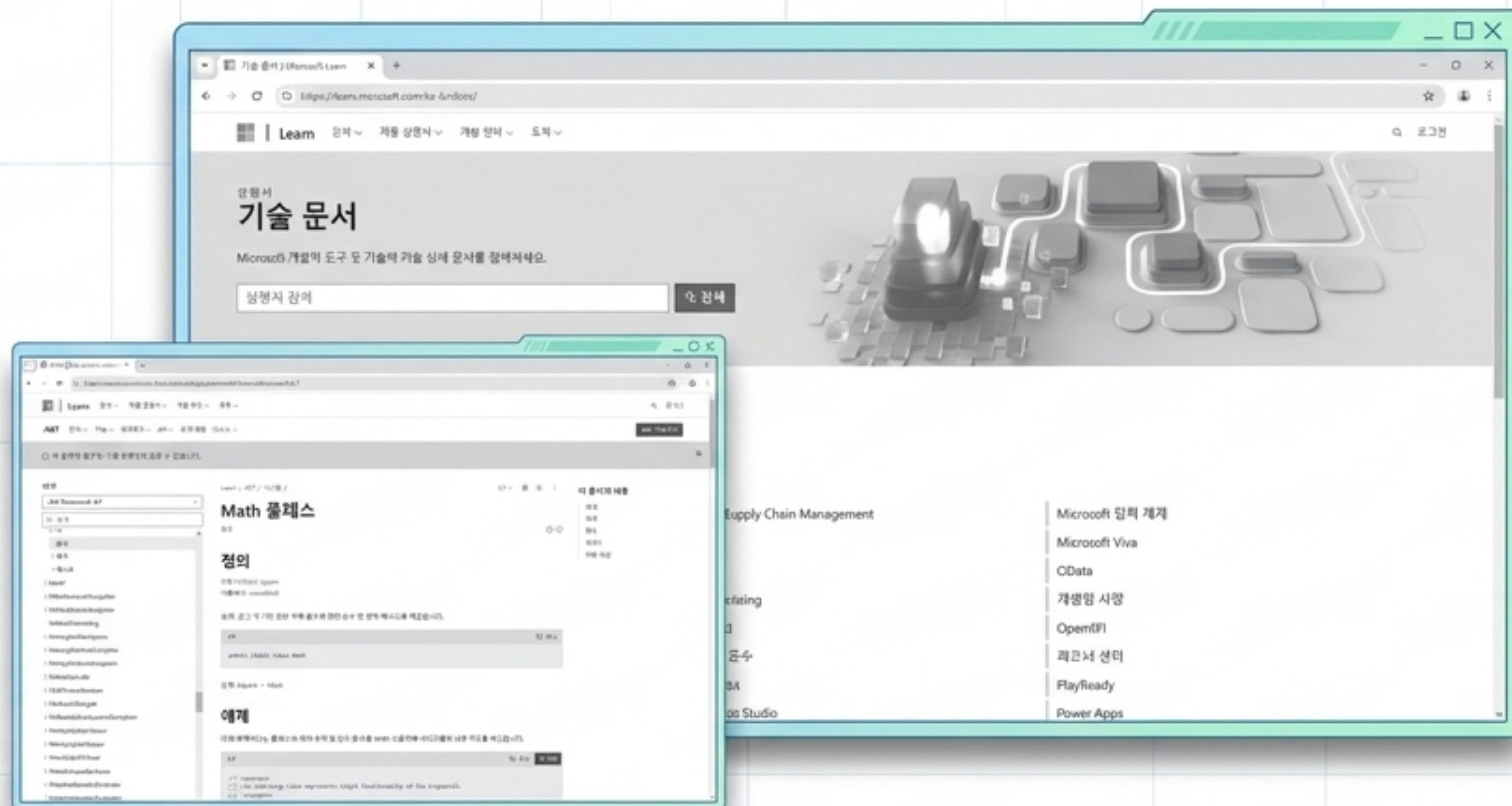
Math처럼 입력값을 받아 즉시 계산 결과만 반환하고, 과거의 기록을 기억할 필요가 없는 경우.

인스턴스 불필요
(클래스 이름으로 직접 메서드 호출)



무한한 C# 생태계를 스스로 항해하는 나침반

- C#에는 상상할 수 없을 만큼 많은 기본 클래스와 메서드가 존재합니다.
- 구글에서 'C# [클래스명]' (예: C# Math 클래스)을 검색하여 Microsoft Learn 공식 기술 문서에 접근하는 습관을 들이세요.



훌륭한 개발자는 모든 코드를 외우는 사람이 아니라, IDE의 자동 완성 기능과 기술 문서를 통해 최적의 도구를 가장 빠르게 찾아 조립하는 사람입니다.